1) Rotar un array hacia la derecha k posiciones
- Descripción: Dado un array de números y un entero k, gira los elementos hacia la derecha k posiciones de manera que se mantenga el orden.

- Solución:

```
function rotateRightInPlace(nums, k) {
  const n = nums.length;
  if (n <= 1) return;
  k = ((k % n) + n) % n;
  if (k === 0) return;

  // inversiones
  const swap = (i, j) => {
    const t = nums[i];
    nums[i] = nums[j];
    nums[j] = t;
  };

  // invertir todos
  for (let i = 0; i < Math.floor(n / 2); i++) swap(i, n - 1 - i);
  // invertir primera k
  for (let i = 0; i < Math.floor(k / 2); i++) swap(i, k - 1 - i);
  // invertir rest
  for (let i = k; i < Math.floor((n + k) / 2); i++) {
    swap(i, n - 1 - (i - k));
  }
}

// Ejemplo:
// let a = [1,2,3,4,5]; rotateRightInPlace(a, 2); // [4,5,1,2,3]
```

2) Subarray con suma máxima.
- Descripción: Encontrar la suma máxima de cualquier subarray contiguo y devolver la suma y el intervalo que define el subarray.

- Solución:

```
function maxSubarraySum(nums) {
  if (nums.length === 0) return {maxSum: 0, start: -1, end: -1};
  let best = -Infinity, cur = 0;
  let bestStart = 0, bestEnd = 0, curStart = 0;

  for (let i = 0; i < nums.length; i++) {
    if (cur <= 0) {
      cur = nums[i];
      curStart = i;
```

IES MARIA ENRIQUEZ

```
    } else {
      cur += nums[i];
    }
    if (cur > best) {
      best = cur;
      bestStart = curStart;
      bestEnd = i;
    }
  }
  return { maxSum: best, start: bestStart, end: bestEnd };
}

// Ejemplo:
// maxSubarraySum([-2,1,-3,4,-1,2,1,-5,4]) -> { maxSum: 6, start: 3, end:
6 }
```

3) Eliminar duplicados en un array ordenado manteniendo el orden.
- Descripción: en un array ordenado, elimina los duplicados y devuelve la longitud que tiene el array generado.

- Solución:

```
function removeDuplicatesSorted(nums) {
  if (nums.length <= 1) return nums.length;
  let write = 1;
  for (let i = 1; i < nums.length; i++) {
    if (nums[i] !== nums[i - 1]) {
      nums[write] = nums[i];
      write++;
    }
  }
  return write; // nueva longitud
}

// Ejemplo:
// let a = [1,1,2,2,3]; let len = removeDuplicatesSorted(a); // a[0..len-1] =
[1,2,3]
```

4) Colocar pares antes de impares manteniendo el orden.
- Descripción: coloca los %2 == 0 al principio manteniendo el orden de los elementos de cada grupo.

- Solución:

```
function stablePartitionEvenBeforeOdd(nums) {
  const temp = [];
  // primera pasada: acumular todo el que es par
```

IES MARIA ENRIQUEZ

```
  for (const x of nums) if (x % 2 === 0) temp.push(x);
  // después, todos los impares
  for (const x of nums) if (x % 2 !== 0) temp.push(x);
  // escribir de nuevo el array original
  for (let i = 0; i < nums.length; i++) nums[i] = temp[i];
}

// Ejemplo:
// let a = [3,1,2,4,5]; stablePartitionEvenBeforeOdd(a); // [2,4,3,1,5]
```

5) Subarray con producto máximo
- Descripción: encontrar la estructura de subarray que tiene el producto máximo; gestiona ceros y negativos de forma iterativa.

- Solución:

```
function maxSubarrayProduct(nums) {
  if (nums.length === 0) return 0;
  let maxProd = nums[0], minProd = nums[0], ans = nums[0];

  for (let i = 1; i < nums.length; i++) {
    const x = nums[i];
    if (x < 0) {
      // intercambiar cuando es negativo
      [maxProd, minProd] = [minProd, maxProd];
    }
    maxProd = Math.max(x, maxProd * x);
    minProd = Math.min(x, minProd * x);
    ans = Math.max(ans, maxProd);
  }
  return ans;
}

// Ejemplo:
// maxSubarrayProduct([2,3,-2,4]) -> 24
```

6) Producto de los elementos sin división (incluir todos menos el propio elemento)
- Descripción: Dado un array de números, obtener un nuevo array donde cada elemento es el producto de todos los otros elementos.
Recomendación: Hacedlo en una pasada iterativa con ayuda de prefijos y sufijos.

IES MARIA ENRIQUEZ

- Solución:

```
function productsExceptSelf(arr) {
  const n = arr.length;
  if (n === 0) return [];
  const left = new Array(n).fill(1);
  const right = new Array(n).fill(1);
  const res = new Array(n);

  // prefijo productos
  for (let i = 1; i < n; i++) {
    left[i] = left[i - 1] * arr[i - 1];
  }

  // sufijo productos
  for (let i = n - 2; i >= 0; i--) {
    right[i] = right[i + 1] * arr[i + 1];
  }

  for (let i = 0; i < n; i++) {
    res[i] = left[i] * right[i];
  }
  return res;
}

// Ejemplo
// console.log(productsExceptSelf([1, 2, 3, 4])); // [24, 12, 8, 6]
```

7) Invertir con recursión (en el mismo array o crea una copia que este invertida)
- Descripción: Invertir un array recursivamente (sin utilizar métodos de array como reverse). Devuelve una nueva copia invertida.

- Solución recursiva:

```
function reverseArrayRecursive(arr) {
  function helper(i, acc) {
    if (i < 0) return acc;
    acc.push(arr[i]);
    return helper(i - 1, acc);
  }
  return helper(arr.length - 1, []);
}

// Ejemplo
// console.log(reverseArrayRecursive([1, 2, 3, 4])); // [4, 3, 2, 1]
```

IES MARIA ENRIQUEZ